Computación cuántica con Qiskit: conceptos fundamentales con código comentado

Genesis Silesky

20 de octubre de 2026

Índice

1. Prerrequisitos y notación **4**
1.1. Números complejos . . . 4
1.2. Vectores, norma y producto interno . . . 4
1.3. Notación de Dirac . . . 4
1.4. Matrices unitarias . . . 4
1.5. Producto tensorial (Kronecker) . . . 4
1.6. Probabilidad básica . . . 4
1.7. Conceptos de Qiskit . . . 5
1.8. Puertas usadas en estos ejemplos . . . 5
1.9. Cómo ejecutar el código . . . 5

2. El qubit como vector **6**
2.1. Objetivo . . . 6
2.2. Conceptos previos . . . 6
2.3. Fundamento teórico . . . 6
2.4. Código (completamente comentado) . . . 6
2.5. Explicación celda por celda . . . 7
2.6. Salida obtenida . . . 7
2.7. Interpretación . . . 7
2.8. Ejercicios . . . 8
2.9. Errores comunes . . . 8

3. Las puertas cuánticas son matrices unitarias **9**
3.1. Objetivo . . . 9
3.2. Conceptos previos . . . 9
3.3. Fundamento teórico . . . 9
3.4. Código (completamente comentado) . . . 9
3.5. Explicación celda por celda . . . 10
3.6. Salida obtenida . . . 10
3.7. Interpretación . . . 10
3.8. Ejercicios . . . 10
3.9. Errores comunes . . . 11

4. Crecimiento exponencial: 2^n amplitudes **12**
4.1. Objetivo . . . 12
4.2. Conceptos previos . . . 12
4.3. Fundamento teórico . . . 12
4.4. Código (completamente comentado) . . . 12

4.5. Explicación celda por celda . . . 13
4.6. Salida obtenida . . . 13
4.7. Interpretación . . . 13
4.8. Ejercicios . . . 13

5. Medición y regla de Born **14**
5.1. Objetivo . . . 14
5.2. Conceptos previos . . . 14
5.3. Fundamento teórico . . . 14
5.4. Código (completamente comentado) . . . 14
5.5. Explicación celda por celda . . . 15
5.6. Salida obtenida . . . 15
5.7. Interpretación . . . 15
5.8. Ejercicios . . . 15

6. Interferencia cuántica **16**
6.1. Objetivo . . . 16
6.2. Conceptos previos . . . 16
6.3. Fundamento teórico . . . 16
6.4. Código (completamente comentado) . . . 16
6.5. Explicación celda por celda . . . 17
6.6. Salida obtenida . . . 17
6.7. Interpretación . . . 17
6.8. Ejercicios . . . 17

7. Entrelazamiento: el estado de Bell **18**
7.1. Objetivo . . . 18
7.2. Conceptos previos . . . 18
7.3. Fundamento teórico . . . 18
7.4. Código (completamente comentado) . . . 19
7.5. Explicación celda por celda . . . 19
7.6. Salida obtenida . . . 19
7.7. Interpretación . . . 19
7.8. Ejercicios . . . 20

8. Algoritmo de Grover: búsqueda cuadrática **21**
8.1. Objetivo . . . 21
8.2. Conceptos previos . . . 21
8.3. Fundamento teórico . . . 21
8.4. Código (completamente comentado) . . . 21
8.5. Explicación celda por celda . . . 22
8.6. Salida obtenida . . . 22
8.7. Interpretación . . . 23
8.8. Ejercicios . . . 23

9. Práctica guiada: Bell, variantes y GHZ **24**
9.1. Objetivo . . . 24
9.2. Conceptos previos . . . 24
9.3. Fundamento teórico . . . 24
9.4. Código (completamente comentado) . . . 24
9.5. Explicación celda por celda . . . 25
9.6. Salida obtenida . . . 25

9.7. Interpretación 25
9.8. Ejercicios 26

10. Ruido y la era NISQ **27**
10.1. Objetivo 27
10.2. Conceptos previos 27
10.3. Fundamento teórico 27
10.4. Código (completamente comentado) 27
10.5. Explicación celda por celda 28
10.6. Salida obtenida 28
10.7. Interpretación 28
10.8. Ejercicios 28

11. Generador de números aleatorios cuántico (QRNG) **29**
11.1. Objetivo 29
11.2. Conceptos previos 29
11.3. Fundamento teórico 29
11.4. Código (completamente comentado) 29
11.5. Explicación celda por celda 30
11.6. Salida obtenida 30
11.7. Interpretación 30
11.8. Ejercicios 30

Referencias **31**

1. Prerrequisitos y notación

Este capítulo reúne las herramientas matemáticas y de software que usan todos los demás. Cada capítulo de concepto indica, en sus cajas azules, qué secciones de aquí necesita; si algo no se entiende, conviene volver a la sección correspondiente antes de continuar.

1.1. Números complejos

Un número complejo es $z = a + bi$ con $a, b \in \mathbb{R}$ e $i^2 = -1$.

- **Conjugado:** $\bar{z} = a - bi$.
- **Módulo:** $|z| = \sqrt{z\bar{z}} = \sqrt{a^2 + b^2}$. Es el que importa para las probabilidades.
- **Forma polar y fórmula de Euler:** $z = |z|e^{i\varphi}$, con $e^{i\varphi} = \cos\varphi + i\sin\varphi$. El ángulo φ es la *fase*.

En Python un complejo se escribe `1+1j` (la unidad imaginaria es `1j`) y `np.abs(z)` devuelve su módulo.

1.2. Vectores, norma y producto interno

Los estados de n qubits viven en $\mathbb{C}^{2^n}$. Para vectores columna $u, v \in \mathbb{C}^d$ el producto interno es $\langle u|v\rangle = \sum_k \bar{u}_k v_k$ y la norma es $\|v\| = \sqrt{\langle v|v\rangle}$. Un estado cuántico válido siempre tiene $\|v\| = 1$ ("normalizado").

1.3. Notación de Dirac

Símbolo	Significado	En Python/numpy
$\|\psi\rangle$	vector columna ("ket")	`np.array([a, b])`
$\langle\psi\|$	su transpuesta conjugada ("bra")	`v.conj()`
$\langle\varphi\|\psi\rangle$	producto interno	`np.vdot(phi, psi)`
$\|0\rangle = \binom{1}{0},\ \|1\rangle = \binom{0}{1}$	base computacional	`Statevector.from_label("0")`

1.4. Matrices unitarias

La adjunta de U es $U^\dagger = \bar{U}^T$ (transpuesta conjugada). U es *unitaria* si $U^\dagger U = UU^\dagger = I$. Las matrices unitarias conservan la norma: $\|Uv\| = \|v\|$, y son invertibles con $U^{-1} = U^\dagger$. Toda puerta cuántica es una matriz unitaria.

1.5. Producto tensorial (Kronecker)

Para combinar sistemas se usa $A \otimes B$. En vectores, $|a\rangle \otimes |b\rangle$ se escribe $|ab\rangle$; por ejemplo $|0\rangle \otimes |1\rangle = |01\rangle = (0, 1, 0, 0)^T$. Con n qubits la base tiene 2^n vectores $|x\rangle$, $x \in \{0, 1\}^n$. Un estado que *no* puede escribirse como $|a\rangle \otimes |b\rangle$ está entrelazado (capítulo 05b).

1.6. Probabilidad básica

- **Regla de Born:** si el estado es $\sum_x c_x |x\rangle$, medir da x con probabilidad $|c_x|^2$ (y esas probabilidades suman 1 por la normalización).
- **Frecuencia y ley de los grandes números:** al repetir N veces un experimento con probabilidad p, la frecuencia observada tiene error típico $\sqrt{p(1-p)/N}$ y tiende a p.

1.7. Conceptos de Qiskit

Circuito (`QuantumCircuit`) Lista ordenada de puertas y mediciones sobre qubits y bits clásicos.

`Statevector` Calcula el vector de estado *exacto* de un circuito sin medir; solo es posible en simulación.

Simulador (`AerSimulator`) Ejecuta circuitos en el computador local, con o sin ruido.

Shot Una ejecución completa del circuito con su medición. Se repite muchas veces para estimar probabilidades.

Conteos (`get_counts`) Diccionario cadena-de-bits → número de veces que salió.

Orden de los bits en Qiskit (little-endian)

Qiskit escribe las cadenas con el qubit 0 a la **derecha**: la cadena `'10'` significa $q_1 = 1$, $q_0 = 0$. En estos ejemplos se señala cada vez que importa.

1.8. Puertas usadas en estos ejemplos

Puerta	Matriz	Efecto	Método Qiskit
H	$\frac{1}{\sqrt{2}}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix}$	superposición	`qc.h(q)`
X	$\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix}$	NOT: $\|0\rangle \leftrightarrow \|1\rangle$	`qc.x(q)`
Z	$\begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix}$	cambia el signo de $\|1\rangle$	`qc.z(q)`
$R_y(\theta)$	$\begin{pmatrix}\cos\frac{\theta}{2} & -\sin\frac{\theta}{2}\\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2}\end{pmatrix}$	rotación real	`qc.ry(theta, q)`
CNOT	(4×4)	invierte el objetivo si el control es $\|1\rangle$	`qc.cx(c, t)`

1.9. Cómo ejecutar el código

Cada ejemplo existe como `.py` (terminal) y como `.ipynb` (Jupyter/Colab). En el `.py`, las líneas `#%%` separan "celdas": en un notebook cada una es una celda; en la terminal no tienen efecto. Instalación (versiones con las que se probó todo):

```
pip install qiskit==2.5.0 qiskit-aer==0.17.2 matplotlib pylatexenc
python 01_qubit_vector.py
```

La API de Qiskit ha cambiado bastante entre versiones; si se usa otra versión algún ejemplo podría fallar.

2. El qubit como vector

2.1. Objetivo

Entender que un qubit es un vector de dos amplitudes complejas normalizado, y cómo esas amplitudes determinan las probabilidades de medición.

2.2. Conceptos previos

Concepto previo: lo que hace falta para este capítulo

Ver el capítulo de prerrequisitos (sección 1): números complejos (módulo y fase), vectores y norma, notación de Dirac y regla de Born. Si solo se recuerda una cosa: el *módulo al cuadrado* de una amplitud es una probabilidad.

2.3. Fundamento teórico

Un bit clásico vale 0 o 1. Un qubit es un vector

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle = \begin{pmatrix}\alpha\\ \beta\end{pmatrix}, \qquad \alpha,\beta\in\mathbb{C}, \quad |\alpha|^2+|\beta|^2 = 1. \tag{1}$$

Por la regla de Born, al medir se obtiene 0 con probabilidad $|\alpha|^2$ y 1 con probabilidad $|\beta|^2$. La condición de normalización asegura que las probabilidades sumen 1. Dos observaciones:

- **Fase global:** $e^{i\gamma}\,|\psi\rangle$ y $|\psi\rangle$ son físicamente indistinguibles.
- **Fase relativa:** la fase de β respecto a α no altera las probabilidades de esta medición, pero sí cómo interfiere el qubit con otros después (capítulo 05a).

Salvo fase global, todo qubit se parametriza como $\cos\frac{\theta}{2}\,|0\rangle + e^{i\varphi}\sin\frac{\theta}{2}\,|1\rangle$ (esfera de Bloch).

Ejemplo del código. Se toma $\alpha = 1/\sqrt{3}$ y $\beta = \sqrt{2/3}\,e^{i\pi/4} = (1+i)/\sqrt{3}$. Entonces $|\alpha|^2 = 1/3$ y $|\beta|^2 = 2/3$, por lo que $P(0) = 1/3$, $P(1) = 2/3$.

2.4. Código (completamente comentado)

```python
# %% [markdown]
# # 01 - El qubit como vector
# |psi> = alpha|0> + beta|1>, con |alpha|^2 + |beta|^2 = 1

# %% Celda 1: importaciones
# numpy nos da numeros complejos y funciones matematicas (sqrt, exp, pi).
import numpy as np

# Statevector representa el estado de un sistema cuantico como un vector
# de amplitudes complejas (el "vector de estado").
from qiskit.quantum_info import Statevector

# %% Celda 2: los estados de la base computacional
# from_label("0") construye |0>, que en forma de vector columna es (1, 0).
# Estos dos vectores son la base canonica: cualquier qubit es una
# combinacion lineal de ellos.
print("|0> =", Statevector.from_label("0").data)  # [1+0j, 0+0j]
print("|1> =", Statevector.from_label("1").data)  # [0+0j, 1+0j]

# %% Celda 3: un qubit arbitrario
```

```python
# Elegimos dos amplitudes complejas alpha y beta.
# alpha es real; beta tiene modulo sqrt(2/3) y una fase de pi/4 radianes.
# La fase e^{i*pi/4} NO cambia la probabilidad de medir, pero si cambia
# como interfiere el qubit con otros (ver el ejemplo 05a).
alpha = 1 / np.sqrt(3)
beta = np.sqrt(2 / 3) * np.exp(1j * np.pi / 4)

# Construimos el estado a partir de la lista de amplitudes [alpha, beta].
psi = Statevector([alpha, beta])

# .data devuelve el arreglo de numpy con las amplitudes.
print("Vector:", psi.data)

# Condicion de normalizacion: la suma de los modulos al cuadrado debe ser 1.
# np.abs de un complejo es su modulo; el 1.0000000000000002 que se imprime
# es error de redondeo de punto flotante, no un error del estado.
print("Norma |a|^2+|b|^2 =", np.sum(np.abs(psi.data) ** 2))

# %% Celda 4: de amplitudes a probabilidades (regla de Born)
# probabilities_dict() aplica la regla de Born: P(k) = |amplitud_k|^2.
# Esperamos P(0) = 1/3 y P(1) = 2/3. Convertimos a float para imprimir limpio.
probs = {str(k): round(float(v), 4) for k, v in psi.probabilities_dict().items()}
print("Probabilidades:", probs)

# %% Celda 5: un vector NO normalizado no es un estado valido
# (1, 1) tiene norma^2 = 2, asi que no representa un qubit fisico.
# is_valid() comprueba justamente que la norma sea 1.
print("(1,1) es valido?:", Statevector([1, 1]).is_valid())  # False
print("(1,1)/sqrt(2) es valido?:", Statevector([1, 1] / np.sqrt(2)).is_valid())  # True
```

2.5. Explicación celda por celda

Celda 1 Importa `numpy` (complejos y funciones) y `Statevector`, la clase que guarda las amplitudes.

Celda 2 Construye $|0\rangle = (1, 0)$ y $|1\rangle = (0, 1)$ con `from_label`. Son la base canónica.

Celda 3 Define α, β y crea el estado. La suma $|\alpha|^2 + |\beta|^2$ sale 1.0000000000000002 y no exactamente 1: es error de redondeo de punto flotante (aritmética de doble precisión).

Celda 4 `probabilities_dict()` aplica la regla de Born. Se convierte a `float` y `str` para que la impresión sea legible (NumPy 2 imprime sus tipos propios).

Celda 5 Muestra que $(1, 1)$ no es válido (norma$^2 = 2$) y que $(1, 1)/\sqrt{2}$ sí lo es.

2.6. Salida obtenida

```
|0> = [1.+0.j 0.+0.j]
|1> = [0.+0.j 1.+0.j]
Vector: [0.57735027+0.j         0.57735027+0.57735027j]
Norma |a|^2+|b|^2 = 1.0000000000000002
Probabilidades: {'0': 0.3333, '1': 0.6667}
(1,1) es valido?: False
(1,1)/sqrt(2) es valido?: True
```

Esta salida es determinista (no hay medición).

2.7. Interpretación

Las amplitudes son números complejos, pero las probabilidades salen reales y suman 1. La fase de β ($\pi/4$) no aparece en las probabilidades: es información que solo se revela mediante interferencia.

2.8. Ejercicios

1. Cambie la fase de β a π y verifique que las probabilidades no cambian.
2. Construya un estado con $P(1) = 0.9$ y compruébelo.
3. ¿Es $(\frac{3}{5}, \frac{4}{5}i)$ un estado válido? Verifíquelo con `is_valid()`.

2.9. Errores comunes

- Olvidar normalizar: Qiskit no lo hace automáticamente y `Statevector` podría ser inválido.
- Confundir amplitud con probabilidad: la amplitud puede ser negativa o compleja; la probabilidad es su módulo al cuadrado.

3. Las puertas cuánticas son matrices unitarias

3.1. Objetivo

Ver que aplicar una puerta equivale a multiplicar una matriz por el vector de estado, y por qué esa matriz debe ser unitaria.

3.2. Conceptos previos

Concepto previo: multiplicación matriz–vector y unitariedad

Ver sección 1: matrices unitarias ($U^\dagger U = I$) y notación de Dirac. Recordatorio: el producto $\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)\left(\begin{smallmatrix} x \\ y \end{smallmatrix}\right) = \left(\begin{smallmatrix} ax+by \\ cx+dy \end{smallmatrix}\right)$.

3.3. Fundamento teórico

Una puerta de un qubit es una matriz $U \in \mathbb{C}^{2\times 2}$ unitaria y el estado evoluciona como $|\psi'\rangle = U|\psi\rangle$. Las más usadas:

$$H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \tag{2}$$

Con $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) =: |+\rangle$ y $H|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) =: |-\rangle$. *(Aquí se usa la notación $|\pm\rangle$ solo como abreviatura.)*

Por qué unitarias. Si $U^\dagger U = I$ entonces $\|U|\psi\rangle\|^2 = \langle\psi|U^\dagger U|\psi\rangle = \||\psi\rangle\|^2$: la normalización (probabilidades que suman 1) se conserva. Además U es invertible con $U^{-1} = U^\dagger$, es decir, la evolución cuántica es *reversible*. Para H se cumple $H^\dagger = H$ y $H^2 = I$.

3.4. Código (completamente comentado)

```python
# %% [markdown]
# # 02 - Las puertas cuanticas son matrices unitarias
# H = 1/sqrt(2) * [[1, 1], [1, -1]]

# %% Celda 1: importaciones
import numpy as np
from qiskit import QuantumCircuit  # un circuito es una lista de puertas
from qiskit.quantum_info import Operator, Statevector
# Operator convierte un circuito (o una puerta) en su matriz unitaria.

# %% Celda 2: la matriz de cada puerta
# Para cada puerta construimos un circuito de 1 qubit con solo esa puerta
# y le pedimos a Qiskit su matriz.
for nombre, puerta in [("H", "h"), ("X", "x"), ("Z", "z")]:
    qc = QuantumCircuit(1)
    # getattr(qc, "h")(0) es lo mismo que qc.h(0): aplica la puerta al qubit 0.
    getattr(qc, puerta)(0)
    # np.round limpia el ruido numerico (0.707106781...476 -> 0.707).
    print(nombre, "=\n", np.round(Operator(qc).data, 3))
# H: crea superposicion. X: "NOT" cuantico (intercambia |0> y |1>).
# Z: deja |0> igual y cambia el signo de |1> (puerta de fase).

# %% Celda 3: aplicar una puerta = multiplicar matriz por vector
H = Operator.from_label("H").data  # matriz 2x2 de Hadamard
ket0 = np.array([1, 0])            # el vector |0>
```

```python
# El operador @ es el producto matricial de numpy.
print("H @ |0> =", H @ ket0)       # (1/sqrt2, 1/sqrt2): superposicion igual

# Qiskit hace exactamente lo mismo con .evolve(): aplica el operador al estado.
evolucionado = Statevector.from_label("0").evolve(Operator.from_label("H"))
print("Statevector:", evolucionado.data)

# %% Celda 4: las puertas son unitarias
# Una matriz U es unitaria si U^dagger U = I, donde U^dagger es la
# transpuesta conjugada (.conj().T). Esto garantiza que la norma del
# vector se conserve: una puerta nunca rompe la normalizacion.
print("H^dagger H = I:", np.allclose(H.conj().T @ H, np.eye(2)))

# Como consecuencia, toda puerta es reversible: su inversa es U^dagger.
# En particular H es su propia inversa (H H = I), lo que se usa en 05a.
print("H H = I:", np.allclose(H @ H, np.eye(2)))
```

3.5. Explicación celda por celda

Celda 1 `Operator` convierte un circuito en su matriz.

Celda 2 Para cada puerta crea un circuito de un qubit y extrae su matriz. `getattr(qc, "h")(0)` equivale a `qc.h(0)`; se usa para recorrer varias puertas en un ciclo.

Celda 3 Calcula $H|0\rangle$ con `@` (producto matricial de NumPy) y con `.evolve()` de Qiskit; ambos dan $(1/\sqrt{2}, 1/\sqrt{2}) \approx (0.707, 0.707)$.

Celda 4 Verifica numéricamente $H^\dagger H = I$ y $H^2 = I$. `np.allclose` compara con tolerancia numérica (no se usa `==` con decimales).

3.6. Salida obtenida

```
H =
 [[ 0.707+0.j  0.707+0.j]
 [ 0.707+0.j -0.707+0.j]]
X =
 [[0.+0.j 1.+0.j]
 [1.+0.j 0.+0.j]]
Z =
 [[ 1.+0.j  0.+0.j]
 [ 0.+0.j -1.+0.j]]
H @ |0> = [0.70710678+0.j 0.70710678+0.j]
Statevector: [0.70710678+0.j 0.70710678+0.j]
H^dagger H = I: True
H H = I: True
```

3.7. Interpretación

Cada puerta es una matriz concreta y calculable. El resultado $H|0\rangle = (0.707, 0.707)$ confirma una superposición con $P(0) = P(1) = \frac{1}{2}$.

3.8. Ejercicios

1. Calcule a mano $XZ|0\rangle$ y $ZX|0\rangle$. ¿Conmutan X y Z?
2. Verifique en el código que X y Z son unitarias.
3. Calcule $H|1\rangle$ y compruebe que sus probabilidades son $\frac{1}{2}, \frac{1}{2}$ aunque una amplitud sea negativa.

3.9. Errores comunes

- Con varios qubits, la matriz de un circuito usa el orden little-endian de Qiskit; para un solo qubit no hay ambigüedad.
- Comparar matrices con `==`: use `np.allclose`.

4. Crecimiento exponencial: 2^n amplitudes

4.1. Objetivo

Cuantificar por qué simular n qubits en una computadora clásica requiere memoria que crece como 2^n.

4.2. Conceptos previos

Concepto previo: producto tensorial y potencias de 2

Ver sección 1. Idea clave: al combinar sistemas con $\otimes$, las dimensiones se *multiplican*: $2 \cdot 2 \cdots 2 = 2^n$. Además, $1\,\text{GiB} = 2^{30}$ bytes.

4.3. Fundamento teórico

El espacio de n qubits es $(\mathbb{C}^2)^{\otimes n} \cong \mathbb{C}^{2^n}$; un estado general es

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} c_x \, |x\rangle \,, \qquad \sum_x |c_x|^2 = 1, \tag{3}$$

con 2^n amplitudes complejas independientes (salvo normalización y fase global). Con `complex128` cada amplitud ocupa 16 bytes, de modo que

$$\text{memoria} = 16 \cdot 2^n \text{ bytes}. \tag{4}$$

Para $n = 30$: $16 \cdot 2^{30}$ B $= 16$ GiB; para $n = 50$: $16 \cdot 2^{50}$ B $= 2^{24}$ GiB $= 16$ PiB, y hay $\approx 1.13 \times 10^{15}$ amplitudes.

Un caso concreto. Aplicar H a cada uno de n qubits partiendo de $|0 \cdots 0\rangle$ da $H^{\otimes n} |0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_x |x\rangle$. Con $n = 4$: 16 amplitudes iguales a $1/4$.

Matiz importante

Esto aplica a la simulación de circuitos *generales* con el vector de estado. Hay circuitos con estructura (p. ej. solo puertas Clifford, o con poco entrelazamiento) que se simulan eficientemente con otros métodos. Tampoco significa que un computador cuántico "guarde" 2^n números accesibles: al medir solo se obtienen n bits.

4.4. Código (completamente comentado)

```python
# %% [markdown]
# # 03 - Con n qubits hay 2^n amplitudes

# %% Celda 1: importaciones
from qiskit import QuantumCircuit
from qiskit.quantum_info import Statevector

# %% Celda 2: memoria necesaria para simular n qubits
# Un estado de n qubits tiene 2^n amplitudes complejas.
# Cada complejo de doble precision (complex128) ocupa 16 bytes
# (8 para la parte real y 8 para la imaginaria).
for n in [1, 2, 10, 20, 30, 40, 50]:
    amplitudes = 2 ** n                 # numero de amplitudes
    gib = amplitudes * 16 / 2 ** 30     # bytes -> GiB (1 GiB = 2^30 bytes)
```

```
    print(f"n={n:>2}  amplitudes={amplitudes:>18,}  memoria ~ {gib:>14,.4f} GiB")
# Con 30 qubits ya son 16 GiB (una laptop buena); con 50, millones de GiB.
# Este crecimiento exponencial es la razon por la que simular es inviable
# a gran escala, y una de las motivaciones de la computacion cuantica.

# %% Celda 3: ver las 2^n amplitudes de un caso pequeno
n = 4
qc = QuantumCircuit(n)
# qc.h(range(n)) aplica Hadamard a cada uno de los n qubits.
# El resultado es la superposicion uniforme de las 2^n cadenas de bits.
qc.h(range(n))
sv = Statevector(qc)  # calcula el vector de estado del circuito
# Todas las amplitudes valen 1/sqrt(2^n) = 1/4 cuando n = 4.
print(f"{len(sv.data)} amplitudes, cada una = {sv.data[0].real:.4f}")
```

4.5. Explicación celda por celda

Celda 2 Recorre varios n, calcula 2^n y convierte a GiB dividiendo entre 2^{30}. Los formatos `:>18,` alinean y ponen separadores de miles.

Celda 3 Construye el estado $H^{\otimes 4}\,|0000\rangle$ y comprueba que hay 16 amplitudes, todas iguales a 0.25.

4.6. Salida obtenida

```
n= 1  amplitudes=                 2  memoria ~         0.0000 GiB
n= 2  amplitudes=                 4  memoria ~         0.0000 GiB
n=10  amplitudes=             1,024  memoria ~         0.0000 GiB
n=20  amplitudes=         1,048,576  memoria ~         0.0156 GiB
n=30  amplitudes=     1,073,741,824  memoria ~        16.0000 GiB
n=40  amplitudes= 1,099,511,627,776  memoria ~    16,384.0000 GiB
n=50  amplitudes=1,125,899,906,842,624  memoria ~ 16,777,216.0000 GiB
16 amplitudes, cada una = 0.2500
```

4.7. Interpretación

Cada qubit extra *duplica* la memoria. A 30 qubits ya se necesitan 16 GiB; unos pocos más y ninguna laptop alcanza. Este es el motivo de que simular sea difícil y, a la vez, parte de la motivación de usar hardware cuántico.

4.8. Ejercicios

1. ¿Cuántos qubits se pueden simular con 64 GiB de RAM?
2. Repita el cálculo con `complex64` (8 bytes por amplitud).
3. Verifique con `Statevector` que para $n = 3$ las amplitudes valen $1/\sqrt{8} \approx 0.354$.

5. Medición y regla de Born

5.1. Objetivo

Comprobar experimentalmente que $P(0) = |\alpha|^2$ y $P(1) = |\beta|^2$, y ver cómo la frecuencia observada converge a esas probabilidades al aumentar los *shots*.

5.2. Conceptos previos

Concepto previo: regla de Born, colapso y frecuencia relativa

Ver sección 1. Dos ideas: (i) medir produce un resultado clásico aleatorio con las probabilidades de Born; (ii) tras medir, el estado *colapsa* al resultado observado, y la superposición original se pierde.

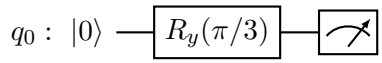
5.3. Fundamento teórico

Para $|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle$, la medición en la base computacional da 0 con probabilidad $|\alpha|^2$ y 1 con $|\beta|^2$. No se puede conocer α, β con una sola medición; hay que repetir el experimento N veces (cada repetición es un *shot*). La frecuencia $\hat{p}$ de "1" tiene error típico

$$\sigma = \sqrt{\frac{p(1-p)}{N}}. \tag{5}$$

Para $p = 0.25$: $\sigma \approx 0.137$ ($N = 10$), 0.043 ($N = 100$), 0.014 ($N = 1000$), 0.0043 ($N = 10000$).

Preparación del estado. La rotación $R_y(\theta)\,|0\rangle = \cos\frac{\theta}{2}\,|0\rangle + \sin\frac{\theta}{2}\,|1\rangle$. Para que $P(1) = \sin^2\frac{\theta}{2} = 0.25$ se necesita $\theta = 2\arcsin(0.5) = \pi/3$.



5.4. Código (completamente comentado)

```python
# %% [markdown]
# # 04 - Medicion: P(0) = |alpha|^2 y P(1) = |beta|^2

# %% Celda 1: importaciones
import sys

import numpy as np
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator  # simulador local de circuitos
from qiskit.visualization import plot_histogram

# La consola de Windows (cp1252) no imprime el dibujo del circuito sin esto.
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

# %% Celda 2: preparar un qubit con P(1) = 0.25
# La rotacion Ry(theta) lleva |0> a cos(theta/2)|0> + sin(theta/2)|1>.
# Queremos sin^2(theta/2) = 0.25, es decir theta = 2*arcsin(sqrt(0.25)).
theta = 2 * np.arcsin(np.sqrt(0.25))

# QuantumCircuit(1, 1): 1 qubit y 1 bit clasico donde guardar la medicion.
qc = QuantumCircuit(1, 1)
qc.ry(theta, 0)     # prepara el estado con las amplitudes deseadas
```

```python
qc.measure(0, 0)    # mide el qubit 0 y guarda el resultado en el bit clasico 0
print(qc.draw())    # dibujo del circuito en texto

# %% Celda 3: repetir la medicion (shots)
# Una sola medicion da un unico 0 o 1 y destruye la superposicion.
# Para estimar las probabilidades hay que repetir el experimento muchas
# veces: cada repeticion se llama "shot".
sim = AerSimulator()
for shots in [10, 100, 1000, 10000]:
    # run() lanza el circuito; result().get_counts() devuelve cuantas veces
    # salio cada resultado, por ejemplo {'0': 749, '1': 251}.
    counts = sim.run(qc, shots=shots).result().get_counts()
    print(shots, "shots ->", counts, f"| freq(1) = {counts.get('1', 0) / shots:.3f}")
# Ley de los grandes numeros: con mas shots, freq(1) se acerca a 0.25.

# %% Celda 4: histograma
# En Colab o Jupyter la figura se muestra sola; en terminal se puede
# guardar con plot_histogram(...).savefig("hist.png").
plot_histogram(sim.run(qc, shots=1000).result().get_counts())
```

5.5. Explicación celda por celda

Celda 1 Además de las importaciones, reconfigura la salida estándar a UTF-8: sin eso la consola de Windows no puede imprimir el dibujo del circuito (error `UnicodeEncodeError`).

Celda 2 Calcula θ, crea un circuito con 1 qubit y 1 bit clásico, aplica R_y y mide.

Celda 3 Ejecuta $10, 100, 1000, 10000$ shots y calcula la frecuencia de "1". `counts.get('1', 0)` evita un error si "1" nunca salió.

Celda 4 Dibuja el histograma (en script puede guardarse con `savefig`).

5.6. Salida obtenida

Los números cambian en cada ejecución

La medición es aleatoria: los conteos exactos varían. Lo que debe reproducirse es la *tendencia*: la frecuencia de "1" se acerca a 0.25 al aumentar los shots.

```
10 shots -> {'1': 3, '0': 7} | freq(1) = 0.300
100 shots -> {'0': 73, '1': 27} | freq(1) = 0.270
1000 shots -> {'0': 728, '1': 272} | freq(1) = 0.272
10000 shots -> {'0': 7501, '1': 2499} | freq(1) = 0.250
```

5.7. Interpretación

Con 10 shots la frecuencia puede alejarse mucho de 0.25 (error típico ≈ 0.14); con 10 000 queda cerca (error ≈ 0.004). La incertidumbre estadística disminuye como $1/\sqrt{N}$.

5.8. Ejercicios

1. Prepare un qubit con $P(1) = 0.9$ y mida 2000 shots.
2. Repita 20 veces la corrida de 100 shots y grafique la dispersión de la frecuencia; compárela con $\sigma = \sqrt{p(1-p)/N}$.
3. ¿Qué obtiene si mide dos veces seguidas el mismo qubit? (Pista: colapso.)

6. Interferencia cuántica

6.1. Objetivo

Mostrar que las amplitudes se suman y pueden cancelarse, y que por eso el resultado de un circuito depende de las *fases* y no solo de las probabilidades intermedias.

6.2. Conceptos previos

Concepto previo: amplitudes con signo y suma de amplitudes

Ver sección 1. Una amplitud puede ser negativa o compleja; una probabilidad no. Al encadenar puertas, las amplitudes que llegan al mismo resultado se *suman* (con su signo/fase) *antes* de elevar al cuadrado.

6.3. Fundamento teórico

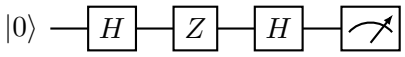
Aplicando H dos veces a $|0\rangle$:

$$H|0\rangle = \tfrac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \tag{6}$$

$$HH|0\rangle = \tfrac{1}{\sqrt{2}}\left[\tfrac{1}{\sqrt{2}}(|0\rangle + |1\rangle) + \tfrac{1}{\sqrt{2}}(|0\rangle - |1\rangle)\right] = |0\rangle\,. \tag{7}$$

La amplitud de $|1\rangle$ es $\frac{1}{2} - \frac{1}{2} = 0$ (interferencia *destructiva*); la de $|0\rangle$ es $\frac{1}{2} + \frac{1}{2} = 1$ (*constructiva*). Un modelo clásico probabilístico (50/50 y luego 50/50) daría de nuevo 50/50, porque las probabilidades nunca se cancelan; las amplitudes sí.

Con una fase en medio, Z cambia el signo de $|1\rangle$: $ZH|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle$ y $H|-\rangle = |1\rangle$. En general $HZH = X$.



Este es el principio de los algoritmos cuánticos: diseñar fases para que las respuestas incorrectas se cancelen y la correcta se refuerce (capítulo 06).

6.4. Código (completamente comentado)

```python
# %% [markdown]
# # 05a - Interferencia
# Las amplitudes se suman o se cancelan. Como H H = I, el |1> se cancela.

# %% Celda 1: importaciones
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Statevector

# %% Celda 2: una H frente a dos H
# Una sola H: superposicion 50/50.
una = QuantumCircuit(1)
una.h(0)
p1 = {str(k): round(float(v), 4) for k, v in Statevector(una).probabilities_dict().items()}
print("1 H :", Statevector(una).data.round(3), p1)

# Dos H seguidas: la segunda H combina las dos amplitudes de la primera.
# La contribucion hacia |1> es (+1/2) + (-1/2) = 0: interferencia
# DESTRUCTIVA. La contribucion hacia |0> es (+1/2) + (+1/2) = 1:
# interferencia CONSTRUCTIVA. El qubit vuelve a |0> con certeza.
```

```python
dos = QuantumCircuit(1)
dos.h(0)
dos.h(0)
p2 = {str(k): round(float(v), 4) for k, v in Statevector(dos).probabilities_dict().items()}
print("2 H :", Statevector(dos).data.round(3), p2)

# %% Celda 3: la fase decide hacia donde interfiere
# La puerta Z cambia el signo de la amplitud de |1> (fase de pi).
# Entre dos H, ese signo invierte la interferencia: H Z H = X,
# asi que el resultado pasa de |0> a |1> con certeza.
fase = QuantumCircuit(1, 1)
fase.h(0)
fase.z(0)
fase.h(0)
fase.measure(0, 0)
# Esperado: {'1': 1000}. Solo cambiamos una fase y cambio el resultado,
# aunque la Z "no hace nada" a las probabilidades por si sola.
print(AerSimulator().run(fase, shots=1000).result().get_counts())
```

6.5. Explicación celda por celda

Celda 2 Compara un circuito con una H (50/50) y otro con dos H (solo $|0\rangle$). Se usa `Statevector` para ver las amplitudes exactas, sin aleatoriedad.

Celda 3 Inserta Z entre las dos H y mide 1000 shots: sale siempre "1". Solo se cambió una fase.

6.6. Salida obtenida

```
1 H : [0.707+0.j 0.707+0.j] {'0': 0.5, '1': 0.5}
2 H : [1.+0.j 0.+0.j] {'0': 1.0}
{'1': 1000}
```

Es determinista porque la medición final tiene probabilidad 1.

6.7. Interpretación

La puerta Z "no hace nada" a las probabilidades de $|+\rangle$, pero cambia por completo el resultado final. Eso solo es posible porque la información de fase se almacena en el signo de las amplitudes.

6.8. Ejercicios

1. Reemplace Z por la puerta S (fase i) y prediga el resultado antes de ejecutarlo.
2. Pruebe $H\,X\,H$: ¿a qué puerta equivale?
3. Explique por qué un "coin flip" clásico repetido no reproduce el resultado de HH.

7. Entrelazamiento: el estado de Bell

7.1. Objetivo

Construir el estado de Bell $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$, demostrar que no se factoriza en dos qubits independientes y observar las correlaciones al medir.

7.2. Conceptos previos

Concepto previo: producto tensorial, estado reducido y entropía

Ver sección 1. Tres ideas nuevas:

- **Estado producto:** $|a\rangle \otimes |b\rangle$; cada qubit tiene su propio estado.
- **Matriz densidad ρ:** describe estados puros y mezclados; para un estado puro $\rho = |\psi\rangle\langle\psi|$.
- **Traza parcial:** "olvidar" un subsistema. El estado del qubit restante es $\rho_0 = \mathrm{Tr}_1\,\rho$. Si ρ_0 es mezclado, el sistema total estaba entrelazado.

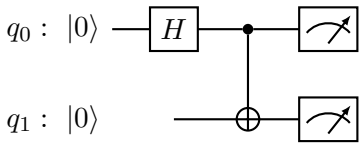
La entropía de von Neumann es $S(\rho) = -\,\mathrm{Tr}(\rho \log_2 \rho)$: vale 0 para un estado puro y 1 bit para un qubit totalmente mezclado ($\rho = I/2$).

7.3. Fundamento teórico

La puerta CNOT (control q_0, objetivo q_1) invierte q_1 si $q_0 = |1\rangle$. Escribiendo los estados como $|q_1 q_0\rangle$ (orden de Qiskit):

$$\mathrm{CNOT}\,|00\rangle = |00\rangle\,, \qquad \mathrm{CNOT}\,|01\rangle = |11\rangle\,. \tag{8}$$

El circuito es



y produce $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$.

No es un producto. Si $|\Phi^+\rangle = (a\,|0\rangle + b\,|1\rangle) \otimes (c\,|0\rangle + d\,|1\rangle) = ac\,|00\rangle + ad\,|01\rangle + bc\,|10\rangle + bd\,|11\rangle$, habría que tener $ad = 0$ y $bc = 0$ pero $ac = bd = \frac{1}{\sqrt{2}} \neq 0$. Si $a = 0$ o $d = 0$ se contradice $ac \neq 0$ o $bd \neq 0$ respectivamente; así que es imposible.

Cuantificarlo. Para $|\Phi^+\rangle$, $\rho_0 = \mathrm{Tr}_1\,\rho = I/2$, de modo que $S(\rho_0) = 1$ bit: el conjunto está en un estado puro bien definido, pero cada parte por separado es máximamente incierta.

Correlación sin comunicación. Al medir, solo salen 00 o 11 (cada uno con probabilidad $\frac{1}{2}$). Cada resultado individual es aleatorio, por lo que no se puede usar el entrelazamiento para enviar un mensaje (*no-signaling*): no hay transmisión de información más rápida que la luz.

Orden de bits

Qiskit escribe $|q_1 q_0\rangle$. Para este estado el orden no importa porque es simétrico.

7.4. Código (completamente comentado)

```
# %% [markdown]
# # 05b - Entrelazamiento: estado de Bell (|00> + |11>)/sqrt(2)

# %% Celda 1: importaciones
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
# partial_trace "olvida" un qubit y deja el estado del otro; entropy mide
# cuanta incertidumbre (mezcla) tiene ese estado reducido.
from qiskit.quantum_info import Statevector, partial_trace, entropy
from qiskit.visualization import plot_histogram

# %% Celda 2: construir el estado de Bell
bell = QuantumCircuit(2)
bell.h(0)        # H sobre el qubit 0: (|0> + |1>)/sqrt(2) en ese qubit
bell.cx(0, 1)    # CNOT: invierte el qubit 1 solo si el qubit 0 vale 1
# Resultado: (|00> + |11>)/sqrt(2). Ojo: Qiskit escribe las cadenas con el
# qubit 0 a la DERECHA (orden little-endian), pero aqui es simetrico.
sv = Statevector(bell)
print("Estado:", sv.data.round(3))  # amplitudes de |00>, |01>, |10>, |11>

# %% Celda 3: prueba de que NO es un producto de dos qubits
# Si el estado fuera producto |a>|b>, cada qubit tendria un estado propio
# bien definido (entropia 0). Aqui, al descartar el qubit 1 el qubit 0
# queda totalmente mezclado: entropia = 1 bit, el maximo para un qubit.
# Eso es entrelazamiento: el todo esta bien definido, las partes no.
rho0 = partial_trace(sv, [1])   # matriz densidad del qubit 0
print("Entropía de entrelazamiento:", entropy(rho0, base=2))  # 1.0

# %% Celda 4: la correlacion al medir
m = bell.copy()
m.measure_all()   # agrega un registro clasico y mide todos los qubits
counts = AerSimulator().run(m, shots=1000).result().get_counts()
# Solo aparecen '00' y '11' (~50% cada uno). Nunca '01' ni '10':
# los resultados estan perfectamente correlacionados, aunque cada uno
# por separado es aleatorio. Esto NO permite enviar informacion
# (no-signaling): el resultado de uno no se puede elegir.
print(counts)
plot_histogram(counts)
```

7.5. Explicación celda por celda

Celda 2 Construye el circuito y obtiene el vector de estado: $(0.707, 0, 0, 0.707)$ para $|00\rangle, |01\rangle, |10\rangle, |11\rangle$.

Celda 3 `partial_trace(sv, [1])` descarta el qubit 1 y `entropy` calcula S: sale 1.0.

Celda 4 Copia el circuito, añade mediciones con `measure_all()` y ejecuta 1000 shots.

7.6. Salida obtenida

```
Estado: [0.707+0.j 0.   +0.j 0.   +0.j 0.707+0.j]
Entropía de entrelazamiento: 1.0
{'11': 503, '00': 497}
```

Los conteos exactos de 00 y 11 varían entre ejecuciones, pero ambos rondan 500 y nunca aparecen 01 ni 10.

7.7. Interpretación

Los resultados de ambos qubits están perfectamente correlacionados aunque cada uno por separado sea aleatorio. Es un recurso que no tiene análogo en probabilidad clásica con variables

locales ocultas (desigualdades de Bell).

7.8. Ejercicios

1. Construya los otros tres estados de Bell ($|\Phi^-\rangle, |\Psi^\pm\rangle$) agregando puertas X y Z al inicio.
2. Calcule la entropía de un estado producto (p. ej. solo H en q_0).
3. Investigue la desigualdad CHSH e impleméntela.

8. Algoritmo de Grover: búsqueda cuadrática

8.1. Objetivo

Implementar Grover para $N = 8$ elementos y comparar el número de consultas con la búsqueda clásica, dejando claro que la ventaja es *cuadrática*, no exponencial.

8.2. Conceptos previos

Concepto previo: oráculo, reflexión y complejidad de consultas

- **Oráculo:** una caja negra O que reconoce la solución. Aquí $O|x\rangle = -|x\rangle$ si x es el elemento marcado y $O|x\rangle = |x\rangle$ si no.
- **Complejidad de consultas:** se cuenta cuántas veces se usa el oráculo, no el tiempo total. Clásico: $O(N)$; Grover: $O(\sqrt{N})$.
- **Interferencia** (capítulo 05a): Grover amplifica la amplitud de la respuesta correcta.

8.3. Fundamento teórico

Se parte de la superposición uniforme $|s\rangle = H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{N}}\sum_x |x\rangle$ con $N = 2^n$. Cada iteración aplica $G = D\,O$, donde $D = 2|s\rangle\langle s| - I$ ("difusión": reflexión respecto al promedio de las amplitudes). Geométricamente G gira el estado, en el plano generado por $|s\rangle$ y la solución $|w\rangle$, un ángulo 2θ con

$$\sin\theta = \frac{1}{\sqrt{N}}. \tag{9}$$

Tras k iteraciones, la probabilidad de medir la solución es

$$P_k = \sin^2\big((2k+1)\theta\big), \qquad k_{\text{opt}} \approx \frac{\pi}{4}\sqrt{N}. \tag{10}$$

Para $N = 8$: $\theta = \arcsin(1/\sqrt{8}) \approx 0.3614$ y $k = \lfloor \frac{\pi}{4}\sqrt{8} \rfloor = 2$, de donde $P_2 = \sin^2(5\theta) \approx 0.945$.

Contraste. Clásicamente, buscar un elemento entre N sin estructura requiere $\sim N/2$ consultas en promedio; Grover usa $\sim \frac{\pi}{4}\sqrt{N}$ (mejora cuadrática, que se sabe óptima para este problema). Esto es muy distinto de Shor (factorización), cuya ventaja sobre los mejores algoritmos clásicos conocidos es *exponencial*.

Oráculo en el código. Para marcar $|101\rangle$ se aplica X a los qubits donde esa cadena tiene un 0 (convirtiéndola en $|111\rangle$), luego un CCZ (hecho como H–Toffoli–H en el último qubit), y se deshacen las X. El CCZ solo cambia el signo cuando los tres qubits valen 1.

8.4. Código (completamente comentado)

```
# %% [markdown]
# # 06 - Grover: busqueda cuadratica O(sqrt(N)) frente a O(N)

# %% Celda 1: importaciones
import numpy as np
from qiskit import QuantumCircuit
# grover_operator arma el operador de Grover G = (difusion)(oraculo).
from qiskit.circuit.library import grover_operator
from qiskit.quantum_info import Statevector

```

```python
# %% Celda 2: el oraculo
# Problema: entre N = 2^3 = 8 cadenas, encontrar la marcada "101".
# El oraculo es una puerta que cambia el signo (fase -1) SOLO de la
# cadena marcada, sin revelar cual es: U|x> = -|x> si x == "101", y |x> si no.
n, marcado = 3, "101"

# Qiskit numera el qubit 0 a la derecha de la cadena, por eso se invierte
# con reversed(). Buscamos los qubits donde la cadena marcada tiene un 0.
ceros = [i for i, b in enumerate(reversed(marcado)) if b == "0"]

oraculo = QuantumCircuit(n)
oraculo.x(ceros)                    # convierte "101" en "111" temporalmente
oraculo.h(n - 1)                    # H - mcx - H sobre el ultimo qubit
oraculo.mcx(list(range(n - 1)), n - 1)  # equivale a un CCZ: fase -1 si todo es 1
oraculo.h(n - 1)
oraculo.x(ceros)                    # deshace el primer paso

# %% Celda 3: aplicar Grover
G = grover_operator(oraculo)        # un paso = oraculo + difusion
# El numero optimo de iteraciones es ~ (pi/4)*sqrt(N). Aqui es 2.
iteraciones = int(np.floor(np.pi / 4 * np.sqrt(2 ** n)))

qc = QuantumCircuit(n)
qc.h(range(n))                      # superposicion uniforme de las 8 cadenas
for _ in range(iteraciones):
    qc.compose(G, inplace=True)     # agrega G al final del circuito
print("Iteraciones de Grover:", iteraciones)

# Cada iteracion amplifica la amplitud de la cadena marcada por interferencia.
probs = Statevector(qc).probabilities_dict()
print("P(", marcado, ") =", round(float(probs[marcado]), 3))  # ~0.945

# %% Celda 4: comparacion de consultas
# Clasico: en promedio hay que revisar N/2 elementos. Grover: ~(pi/4)*sqrt(N).
# La ventaja es cuadratica (no exponencial como en Shor).
for N in [8, 1_000, 1_000_000, 10**12]:
    print(f"N={N:>16,}  clásico ~ {N // 2:>16,}  Grover ~ {int(np.pi / 4 * np.sqrt(N)):>10,}")
```

8.5. Explicación celda por celda

Celda 2 Construye el oráculo. `reversed(marcado)` se usa porque Qiskit numera el qubit 0 a la derecha. `mcx` es una puerta X con varios controles.

Celda 3 `grover_operator` arma G a partir del oráculo. Se calcula k_{opt}, se aplica H a todos y se repite G k veces. `Statevector` da la probabilidad exacta de `"101"`.

Celda 4 Tabla comparativa de consultas: la ventaja crece con N pero solo como raíz.

8.6. Salida obtenida

```
Iteraciones de Grover: 2
P( 101 ) = 0.945
N=               8  clásico ~               4  Grover ~          2
N=           1,000  clásico ~             500  Grover ~         24
N=       1,000,000  clásico ~         500,000  Grover ~        785
N=1,000,000,000,000  clásico ~ 500,000,000,000  Grover ~    785,398
```

Coincide con la teoría: $P_2 \approx 0.945$.

8.7. Interpretación

Con $N = 10^{12}$ Grover necesita unas 7.9×10^5 consultas frente a 5×10^{11}: una mejora enorme, pero cuadrática. Además cada consulta cuántica puede ser mucho más cara que una clásica, y el oráculo debe construirse como un circuito; por eso la ventaja práctica depende del problema y del hardware.

8.8. Ejercicios

1. Cambie `marcado` a `"010"` y verifique el resultado.
2. Use 1 y 3 iteraciones en lugar de 2: ¿qué pasa con P_k? (Pista: es una función periódica de k; "pasarse" empeora el resultado.)
3. Extienda a $n = 4$ y calcule el k_{opt}.

9. Práctica guiada: Bell, variantes y GHZ

9.1. Objetivo

Ejecutar el circuito de Bell, predecir resultados antes de verlos, modificarlo para distinguir superposición de determinismo, y generalizarlo a tres qubits (estado GHZ).

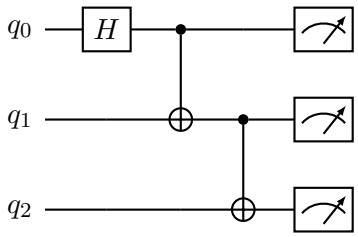
9.2. Conceptos previos

Concepto previo: entrelazamiento y shots

Haber leído el capítulo 05b (estado de Bell) y el 04 (shots y frecuencia).

9.3. Fundamento teórico

- **Bell:** H en q_0 y CNOT $q_0 \to q_1$ dan $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. Se esperan $\approx 50\,\%$ de 00 y $\approx 50\,\%$ de 11.
- **Variante con X:** $X|0\rangle = |1\rangle$ no crea superposición; el CNOT copia ese 1, así que el estado es $|11\rangle$ y el resultado es determinista. Esto aísla el papel de H: es la fuente de la aleatoriedad.
- **GHZ:** con H y dos CNOT en cadena se obtiene $\frac{1}{\sqrt{2}}(|000\rangle + |111\rangle)$.



9.4. Código (completamente comentado)

```python
# %% [markdown]
# # 07 - Practica guiada: Bell, variantes y reto GHZ

# %% Celda 1: importaciones y simulador
import sys

from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram

# La consola de Windows (cp1252) no imprime el dibujo del circuito sin esto.
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

sim = AerSimulator()  # un solo simulador reutilizado en todas las celdas

# %% Celda 2: circuito de Bell
qc = QuantumCircuit(2, 2)   # 2 qubits y 2 bits clasicos
qc.h(0)                     # superposicion en el qubit 0
qc.cx(0, 1)                 # CNOT: entrelaza el qubit 1 con el 0
qc.measure([0, 1], [0, 1])  # qubit i -> bit clasico i
print(qc.draw())
# Esperado: ~50% '00' y ~50% '11', y casi nunca '01' ni '10'.
```

```python
print(sim.run(qc, shots=1000).result().get_counts())

# %% Celda 3: cambiar h(0) por x(0)
# X es el NOT: lleva |0> a |1> sin crear superposicion. El CNOT copia
# ese 1 al qubit 1, asi que el resultado es determinista: siempre '11'.
# Moraleja: el aleatorio de la celda 2 venia de la superposicion de H.
qx = QuantumCircuit(2, 2)
qx.x(0)
qx.cx(0, 1)
qx.measure([0, 1], [0, 1])
print(sim.run(qx, shots=1000).result().get_counts())

# %% Celda 4: reto - estado GHZ de 3 qubits
# Generaliza Bell: (|000> + |111>)/sqrt(2). Cada CNOT "propaga" el valor
# del qubit anterior al siguiente.
ghz = QuantumCircuit(3, 3)
ghz.h(0)
ghz.cx(0, 1)
ghz.cx(1, 2)
ghz.measure([0, 1, 2], [0, 1, 2])
print(sim.run(ghz, shots=1000).result().get_counts())  # solo '000' y '111'

# %% Celda 5: reto - efecto del numero de shots
# Con pocos shots el histograma es muy irregular (ej. 8 contra 2);
# con muchos se acerca al 50/50 teorico. Es ruido estadistico, no fisico.
for shots in [10, 100, 10000]:
    print(shots, sim.run(qc, shots=shots).result().get_counts())
plot_histogram(sim.run(qc, shots=10000).result().get_counts())
```

9.5. Explicación celda por celda

Celda 2 Bell con mediciones; `measure([0,1],[0,1])` conecta cada qubit con un bit clásico.

Celda 3 Sustituye `h(0)` por `x(0)`.

Celda 4 Reto: GHZ de 3 qubits.

Celda 5 Reto: distintos números de shots.

9.6. Salida obtenida

```
{'11': 506, '00': 494}
{'11': 1000}
{'000': 503, '111': 497}
10 {'11': 8, '00': 2}
100 {'00': 51, '11': 49}
10000 {'00': 5003, '11': 4997}
```

(El dibujo de texto del circuito, que imprime el script, no se reproduce aquí.)

9.7. Interpretación

- Con x en lugar de h solo sale 11: ya no hay aleatoriedad.
- GHZ solo da 000 y 111.
- Con 10 shots, las proporciones pueden alejarse mucho de 50/50 (aquí, 8 contra 2); con 10 000 se acercan. Es ruido *estadístico*, no físico (el siguiente capítulo introduce el ruido físico).

Pregunta clave: ¿por qué nunca sale 01 ni 10? Porque el CNOT fuerza que q_1 copie el valor de q_0 en la base computacional.

9.8. Ejercicios

1. Prediga y verifique qué ocurre con $h(0)$ seguido de $x(1)$ (sin CNOT).
2. Extienda GHZ a 5 qubits.
3. Mida solo uno de los tres qubits de GHZ y observe cuál es la distribución del resto.

10. Ruido y la era NISQ

10.1. Objetivo

Ver cómo los errores de puertas hacen aparecer resultados que en un circuito ideal serían imposibles (01 y 10 en un estado de Bell).

10.2. Conceptos previos

Concepto previo: canal de ruido y NISQ

- **Canal de ruido:** una operación que describe cómo un error transforma un estado. El canal *depolarizante* sobre k qubits es $\rho \mapsto (1-\lambda)\rho + \lambda \frac{I}{2^k}$: con probabilidad λ el estado se reemplaza por el estado máximamente mezclado.
- **NISQ** (*Noisy Intermediate-Scale Quantum*, Preskill 2018): dispositivos con decenas a pocos cientos de qubits, con errores, sin corrección de errores a escala.
- **Corrección de errores:** codifica un qubit *lógico* en muchos qubits *físicos*; el costo (cientos o más por qubit lógico, según el código y la tasa de error) es una barrera actual.

10.3. Fundamento teórico

El modelo del código aplica error depolarizante de 1 % tras cada H y de 5 % tras cada CNOT. Las puertas de dos qubits tienen más error que las de uno, lo cual es cualitativamente cierto en hardware real. **Estos valores son ilustrativos y no corresponden a ningún dispositivo.**

Con ruido, el estado de salida deja de ser el estado puro de Bell y las mediciones producen también 01 y 10 con probabilidad pequeña. En la corrida mostrada aparecieron 42 de 2000 shots (2.1 %) con paridad incorrecta.

10.4. Código (completamente comentado)

```
# %% [markdown]
# # 08 - Era NISQ: el ruido produce los resultados 01 y 10

# %% Celda 1: importaciones
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
# NoiseModel agrupa los errores; depolarizing_error(p, k) es un error
# que, con probabilidad p, reemplaza el estado de k qubits por uno aleatorio.
from qiskit_aer.noise import NoiseModel, depolarizing_error
from qiskit.visualization import plot_histogram

# %% Celda 2: circuito y modelo de ruido
# El mismo circuito de Bell de los ejemplos anteriores.
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.measure([0, 1], [0, 1])

ruido = NoiseModel()
# Error de 1% en cada puerta H (puertas de 1 qubit: errores chicos).
ruido.add_all_qubit_quantum_error(depolarizing_error(0.01, 1), ["h"])
# Error de 5% en cada CNOT. En hardware real las puertas de 2 qubits
# son bastante peores que las de 1, por eso se les da mas error.
ruido.add_all_qubit_quantum_error(depolarizing_error(0.05, 2), ["cx"])
```

```
# Los valores son ilustrativos, NO miden ningun equipo concreto.

# %% Celda 3: comparar ideal y ruidoso
ideal = AerSimulator().run(qc, shots=2000).result().get_counts()
# Pasar noise_model hace que el simulador inyecte los errores al ejecutar.
noisy = AerSimulator(noise_model=ruido).run(qc, shots=2000).result().get_counts()
print("Ideal  :", ideal)    # solo 00 y 11
print("Ruidoso:", noisy)    # aparecen 01 y 10: errores que rompen la correlacion
# Este es el panorama NISQ (Noisy Intermediate-Scale Quantum): pocos qubits
# y con errores, lo que limita la profundidad de los circuitos utiles.
plot_histogram([ideal, noisy], legend=["ideal", "ruidoso"])
```

10.5. Explicación celda por celda

Celda 2 Mismo circuito de Bell; se crea un `NoiseModel` y se asocian errores a las puertas `h` y `cx`.

Celda 3 Se ejecuta el circuito en un simulador ideal y en uno con ruido (`noise_model=...`) y se comparan los conteos.

10.6. Salida obtenida

```
Ideal  : {'00': 996, '11': 1004}
Ruidoso: {'11': 993, '00': 960, '01': 21, '10': 26}
```

Los números exactos cambian en cada ejecución.

10.7. Interpretación

El ruido rompe la correlación perfecta. En un equipo real también hay errores de lectura, decoherencia, etc., que este modelo simple no incluye. Comparar este histograma con uno obtenido en hardware real (p. ej. IBM Quantum o las SpinQ) es la mejor forma de ilustrar la era NISQ.

10.8. Ejercicios

1. Aumente el error del CNOT a 20 % y observe el cambio.
2. Mida la fidelidad aproximada como la fracción de shots en 00 o 11.
3. Investigue el error de lectura (`ReadoutError`) y agréguelo.

11. Generador de números aleatorios cuántico (QRNG)

11.1. Objetivo

Generar bits aleatorios midiendo qubits en superposición y usarlos para elegir la jugada de un bot de Tic-Tac-Toe, señalando las limitaciones de la simulación.

11.2. Conceptos previos

Concepto previo: aleatoriedad, uniformidad y módulo

- **Aleatoriedad pseudoaleatoria vs. física:** un PRNG es un algoritmo determinista a partir de una semilla; una fuente física (p. ej. la medición de un qubit real) no lo es.
- **Uniformidad:** todos los resultados posibles tienen igual probabilidad.
- Medición y regla de Born (capítulo 04).

11.3. Fundamento teórico

$H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$: medir da 0 o 1 con probabilidad $\frac{1}{2}$. Con n qubits en paralelo se obtienen n bits independientes $\rightarrow$ un entero uniforme en $\{0, \ldots, 2^n - 1\}$.

Sesgo por módulo. Para elegir entre m opciones con x mód m la distribución solo es uniforme si m divide a 2^n. En el código se eligen 4 bits (16 valores) para $m = 5$ casillas; $16 = 3 \cdot 5 + 1$, así que la casilla 0 sale con probabilidad $4/16$ y las demás con $3/16$. Una solución es el *muestreo por rechazo*: descartar los valores ≥ 15 y repetir.

La advertencia importante. `AerSimulator` calcula las probabilidades y luego toma muestras con un generador pseudoaleatorio. Por lo tanto, **en simulación los bits no son aleatoriedad física**. Solo ejecutando en hardware real la fuente es física (aun así, el ruido del equipo puede introducir sesgo y se recomienda post-procesamiento/extractores).

11.4. Código (completamente comentado)

```python
# %% [markdown]
# # 09 - Aleatoriedad cuantica (QRNG), conecta con el Tic-Tac-Toe

# %% Celda 1: importaciones
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator

# %% Celda 2: generador de bits
def bits_cuanticos(n_bits: int) -> str:
    """Devuelve una cadena de n_bits bits generada midiendo n_bits qubits en
    superposicion. Con H, cada qubit da 0 o 1 con probabilidad 1/2."""
    qc = QuantumCircuit(n_bits, n_bits)
    qc.h(range(n_bits))                        # cada qubit en (|0> + |1>)/sqrt(2)
    qc.measure(range(n_bits), range(n_bits))
    # memory=True guarda el resultado de CADA shot, no solo el conteo.
    # Con shots=1 obtenemos una unica cadena, p.ej. '11011110'.
    return AerSimulator().run(qc, shots=1, memory=True).result().get_memory()[0]

# %% Celda 3: usar los bits
bits = bits_cuanticos(8)
```

```
print("Bits:", bits, "-> entero:", int(bits, 2))  # int(..., 2) lee base 2

# Elegir una casilla libre del tablero 3x3 (indices 0-8).
# El modulo (%) mapea el entero a un indice valido de la lista.
# Nota: 16 valores % 5 casillas no es perfectamente uniforme (leve sesgo);
# para un uso serio se descartan los valores sobrantes y se repite.
libres = [0, 2, 3, 5, 8]
print("Jugada del bot:", libres[int(bits_cuanticos(4), 2) % len(libres)])

# IMPORTANTE: AerSimulator usa un generador pseudoaleatorio. Solo en
# hardware real la aleatoriedad es de origen fisico.
```

11.5. Explicación celda por celda

Celda 2 La función arma un circuito con n qubits, aplica H a todos y mide. `shots=1, memory=True` devuelve la cadena de esa única ejecución.

Celda 3 Convierte los bits en un entero (`int(bits, 2)`) y elige una casilla libre del tablero con el operador módulo.

11.6. Salida obtenida

```
Bits: 10100010 -> entero: 162
Jugada del bot: 3
```

Distinta en cada ejecución (es el punto del generador).

11.7. Interpretación

Se ilustra el vínculo entre el concepto (medición) y una aplicación concreta (un bot que elige jugadas). El ejemplo también expone una buena práctica de ingeniería: validar que el mapeo de bits a opciones no introduzca sesgo.

11.8. Ejercicios

1. Implemente el muestreo por rechazo y verifique con 10 000 jugadas que las 5 casillas tienen frecuencia $\approx 20\,\%$.
2. Mida con una prueba de χ^2 la uniformidad de 10 000 bits.
3. Cambie el simulador por un backend real (sin ejecutar en la charla) y compare.

Referencias

- M. A. Nielsen e I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary Edition, Cambridge University Press, 2010. (Referencia general; cap. 1 y 2 para los fundamentos.)
- Documentación de Qiskit y el curso *Basics of quantum information* de IBM Quantum Learning (verificar la versión vigente de la API antes de usarla: cambia entre versiones).
- M. Born, "Zur Quantenmechanik der Stoßvorgänge", *Zeitschrift für Physik* 37, 863–867, 1926. (Regla de Born.)
- J. S. Bell, "On the Einstein Podolsky Rosen paradox", *Physics* 1, 195–200, 1964.
- D. M. Greenberger, M. A. Horne y A. Zeilinger, "Going beyond Bell's theorem", en *Bell's Theorem, Quantum Theory and Conceptions of the Universe*, Kluwer, 1989. (Estados GHZ.)
- L. K. Grover, "A fast quantum mechanical algorithm for database search", *Proc. 28th ACM STOC*, 212–219, 1996.
- P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring", *Proc. 35th IEEE FOCS*, 124–134, 1994.
- J. Preskill, "Quantum Computing in the NISQ era and beyond", *Quantum* 2, 79, 2018.